

DistAL: Distance-based Advantage Learning for VLA Fine-Tuning

Reece O’Mahoney¹ and Ioannis Havoutis¹

Abstract—Vision-language-action models (VLAs) have transformed the field of robotic manipulation in recent years by combining the semantic understanding of LLMs with the precise control of flow-matching policies. Advantage conditioning is a recent technique that iteratively improves VLAs by training a value function on deployment data and using this to train an advantage-conditioned policy. Previous works have only applied simple, low-information success/failure rewards, which leave the value function unable to distinguish states of differing quality beyond how far along the task they appear. Motivated by an exploration of out-of-distribution (OOD) detection methods, we introduce Distance-based Advantage Learning (DistAL), which, by using an embedding space distance as a reward, produces a more informative value function and subsequently a higher downstream task success rate. We validate our method on a series of simulation benchmarks and dexterous bi-manual manipulation tasks on real hardware.

I. INTRODUCTION

VLAs [1]–[4] have allowed for a level of semantic generalisation and dexterous manipulation not possible with previous generations of robotic control techniques. In particular, the rise of flow-matching [3]–[5] experts for generating continuous actions, as opposed to earlier models which discretised actions into tokens [6], typically predicted auto-regressively [1], [2], has greatly enhanced the ability of transformer-based policies to follow complex action distributions accurately without drifting out of distribution. However, given the uncertainty of the dynamics in contact-rich, dexterous manipulation tasks, the policies can still fail to achieve the kind of success rates that would make them viable for real-world deployment. In response to this, the robotics community has been actively developing strategies for improving robotic performance from their own deployment data [7], [8].

One popular and successful approach has been via the use of intervention data [7], [9], [10], where a human operator takes over control from the policy when, or preferably just before, it reaches some failure state. While this has proven successful, it significantly increases the hardware and operational complexity needed to develop a reliable model, as well as requiring an expert teleoperator to be ready to intervene on potentially dozens of hours of trials. Using online reinforcement learning to fine-tune VLAs, either directly [11]–[15] or by distilling RL specialists into them [16], has been another popular direction, but suffers from issues of poor sample efficiency, instability, and extremely high computational cost given the large parameter counts of the base models. Recent work has improved sample efficiency [17], but

still depends on human-provided rewards and interventions during training.

Recently, advantage conditioning [8], [18] has emerged as a promising new direction for leveraging deployment data. It works by training a value function on the data, which is then used to compute the advantage at each timestep. By thresholding a certain advantage level as “positive”, and optionally sharpening the conditioning with classifier-free guidance [19], this method is essentially offline RL. This does not require an expert human teleoperator like intervention-based methods, and also does not suffer from the same instability and sample efficiency issues as online RL.

Previous methods have only applied this technique to simple success/failure rewards [8], where a constant per-step penalty means the only factor that raises a state’s value is the estimated number of steps remaining. States are therefore ranked purely by how far along the task they appear, with no regard for any other signal in the observations. For instance, an unstable grasp that barely holds an object and a secure one receive the same value despite very different odds of eventual success. This relationship is somewhat captured by the failure penalty, but the credit assignment is noisy and indirect. Custom, task-specific reward functions could be used instead, but this comes with a substantial cost to scalability, as the form and weighting of these rewards require laborious tuning and/or manual frame-by-frame labelling.

We present *Distance-based Advantage Learning* (DistAL) as an alternative to both of these. DistAL is a drop-in replacement for the reward signal: it slots directly into the advantage-conditioning pipeline of [8] with the policy, value function and training procedure left unchanged, isolating reward design as the single variable in our comparisons. Our approach is motivated by the idea that failure trajectories will be OOD when compared to a reference dataset of successes. By exploring the performance of different OOD detection methods at predicting failure, we found that a *kNN distance in image embedding space* was most effective, both at the trajectory level and at the per-timestep level that a value function actually consumes, and thus use it as a reward function. Compared to a binary success label, it reflects the *immediate* quality of each observation rather than relying on the failure being propagated backwards in time through the value function. Our experimental evaluation shows that downstream policies trained with this value function for advantage conditioning exhibit improved performance on a range of simulated and real tasks, with the largest gains where success and failure are visually separable.

In summary, our contributions are:

- 1) A comparison of out-of-distribution detection methods

¹The authors are with the Oxford Robotics Institute, University of Oxford, UK. {reeceo, ioannis}@robots.ox.ac.uk

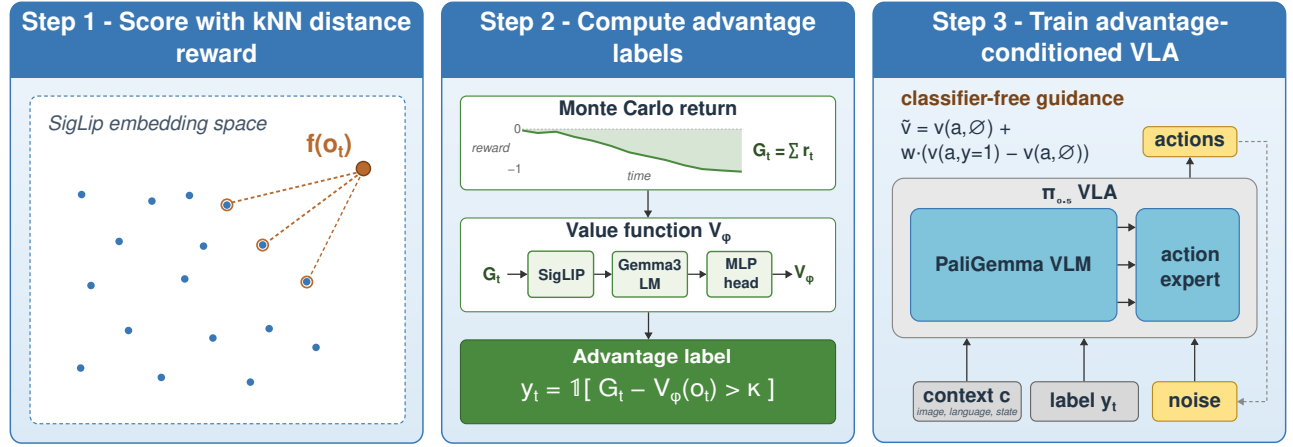


Fig. 1. Overview of DistAL. Instead of the sparse success/failure reward used by prior advantage-conditioning methods, we score each deployment observation by its k -nearest-neighbours distance to the base VLA’s training distribution in SigLIP feature space. The resulting dense reward yields a more informative value function and, in turn, a stronger advantage-conditioned policy.

as predictors of VLA task failure, at both the trajectory and per-timestep level, identifying kNN distance in the base VLA’s SigLIP feature space as the strongest signal.

- 2) A controlled study of using this distance as a dense reward for advantage conditioning, holding the rest of the pipeline fixed, including an analysis of when the underlying assumption holds and when it does not.
- 3) Validation on the LIBERO and LIBERO-plus simulated benchmark suites and on two dexterous bi-manual manipulation tasks on real robot hardware.

II. RELATED WORK

Vision-language-action models. VLAs extend pre-trained vision-language backbones with action prediction heads, inheriting the semantic priors of internet-scale pre-training. Early models such as RT-1 [6] discretised actions into tokens, and RT-2 [1] and OpenVLA [2] predict these tokens autoregressively, which is too slow for high-frequency control and performs poorly on bimanual tasks; OpenVLA-OFT [20] mitigates this with parallel decoding. The dominant approach more recently has been to generate continuous action chunks with a diffusion or flow-matching head [21]–[23], increasingly coupled with a pre-trained VLM backbone [3], [4], [24], [25]. These policies remain imitation-learned from demonstrations and cannot improve from their own interactions, motivating the work below on learning from deployment data.

Improving VLAs from deployment data. A first family exploits human interventions during deployment: HIL-SERL [7] combines interventions with online RL, while Sirius [9] and Sirius-Fleet [10] re-weight or solicit interventions using trust and anomaly signals, requiring an expert teleoperator throughout collection. A second family applies online RL to VLAs [11]–[15], or distils RL-trained specialists into them [16], contending with sample inefficiency, instability, and substantial compute cost. A third family, the one DistAL belongs to, uses offline value learning on logged trajectories. Building on offline-RL foundations [26]–[29], recent works

train a value function on logged data and use this to improve the VLA. *Steering Your Generalists* [30] re-ranks candidate actions by value, and $\pi_{0.6}^*/\text{RECAP}$ [8] conditions a flow-matching VLA on a binary advantage label, optionally sharpened with classifier-free guidance [19]. Both rely on sparse success/failure rewards that make credit assignment more difficult. DistAL targets this weakness by replacing the binary signal with a dense, embedding-distance reward.

Failure and out-of-distribution detection for policies.

A separate literature recognises when a learned policy is failing or operating out of distribution: anomaly scores in LLM-embedding space [31], temporal-consistency and VLM progress checks [32], value functions reused as plan-feasibility scores [33], label-free uncertainty signals [34], and calibrated planner uncertainty [35]. These share our observation that learned representations carry strong signal about eventual success. Methodologically, our reward draws on the OOD-detection community: feature-space nearest-neighbour distance [36] and class-conditional Mahalanobis distance [37] score test points by distance to the training distribution, with deep ensembles [38] as a generic uncertainty backbone. DistAL borrows this distance-to-training-set signal but, rather than using it for runtime rejection, repurposes it as a per-timestep reward for offline value learning.

III. PRELIMINARIES

Flow-matching VLAs. VLAs predict a chunk of H continuous actions $a = (a^1, \dots, a^H) \in \mathbb{R}^{H \times d_a}$ from an observation–language context $c = (o, \ell)$ using a conditional flow-matching head [3], [5]. The head learns a time-dependent velocity field $v_\theta(a_\tau, \tau, c)$ that transports a Gaussian noise sample $a_0 \sim \mathcal{N}(0, I)$ to a data sample a_1 along the linear interpolation $a_\tau = (1 - \tau)a_0 + \tau a_1$ for flow time $\tau \in [0, 1]$. The training objective is

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{\tau, a_0, (a_1, c) \sim \mathcal{D}} \left[\|v_\theta(a_\tau, \tau, c) - (a_1 - a_0)\|^2 \right]. \quad (1)$$

At inference an action chunk is produced by integrating $\dot{a} = v_\theta(a, \tau, c)$ from $a_0 \sim \mathcal{N}(0, I)$ over $\tau \in [0, 1]$ with a

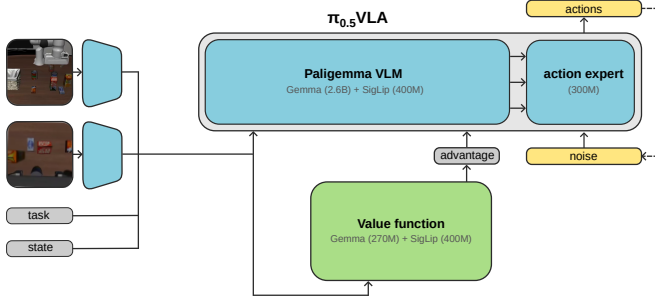


Fig. 2. DistAL architecture from Section IV-B. The value function V_ϕ is trained on deployment data using the kNN feature-distance reward, and the resulting advantage label is injected into the VLA via classifier-free guidance.

small number of Euler steps, typically between 5 and 10.

Advantage conditioning via classifier-free guidance.

Given a dataset of deployment trajectories $\mathcal{D}_{\text{deploy}} = \{(o_t, a_t, r_t)\}_{t=0}^T$ annotated with per-timestep rewards r_t , advantage conditioning [8] first fits a state-value function V_ϕ by Monte Carlo regression onto the empirical return (the original uses an N -step bootstrapped estimate of the advantage; we use the full-episode return),

$$\mathcal{L}_V(\phi) = \mathbb{E}_{(o_t, G_t) \sim \mathcal{D}} \left[(V_\phi(o_t) - G_t)^2 \right], \quad G_t = \sum_{k=0}^{T-t} r_{t+k}, \quad (2)$$

and converts the resulting Monte Carlo advantage $\hat{A}_t = G_t - V_\phi(o_t)$ into a conditioning label $y_t = \mathbb{1}[\hat{A}_t > \kappa]$ via a threshold κ . The VLA is then fine-tuned with classifier-free guidance [19] by dropping the advantage label with probability p , which we represent as a null token $\emptyset$. At inference, the velocity field is steered toward positive advantage actions via

$$\begin{aligned} \tilde{v}_\theta(a_\tau, \tau, c, y=1) &= v_\theta(a_\tau, \tau, c, \emptyset) \\ &+ w \cdot (v_\theta(a_\tau, \tau, c, y=1) \\ &- v_\theta(a_\tau, \tau, c, \emptyset)), \end{aligned} \quad (3)$$

with guidance weight $w > 1$.

IV. METHOD

The advantage-conditioning pipeline of Section III has so far been paired only with a binary success/failure reward [8], which, as noted above, ranks states by estimated steps remaining and chance of success. DistAL replaces this constant per-step penalty with a dense reward based on a k -nearest-neighbours distance in the vision embedding space. We describe the score calculation and encoder choice (Section IV-A) and how the score plugs into the rest of the pipeline (Section IV-C).

A. kNN feature-distance reward

Let $f : \mathcal{O} \rightarrow \mathbb{R}^{d_f}$ be a frozen image encoder and let $\mathcal{I} = \{f(o^{(i)})\}_{i=1}^N$ denote an index of embeddings computed over the observations in the dataset $\mathcal{D}_{\text{train}}$ used to train the base VLA. For any observation o encountered at deployment,

we define its k -nearest-neighbours distance to the training distribution as

$$s_k(o) = \frac{1}{k} \sum_{j \in \mathcal{N}_k(f(o); \mathcal{I})} \|f(o) - z_j\|_2, \quad (4)$$

where $\mathcal{N}_k(z; \mathcal{I})$ returns the indices of the k nearest neighbours of z in $\mathcal{I}$ under the Euclidean norm and $z_j \in \mathcal{I}$ is the j th indexed embedding. Observations close to the support of the training distribution receive low scores; observations that drift away from it receive high scores.

We then take the per-timestep reward used to drive the value function in (2) to be the negative distance,

$$r_t = -s_k(o_t) \quad (5)$$

normalised to $[-1, 0]$. This reward encodes the assumption that observations close to the base VLA's training distribution are better. That assumption is not universally true: a policy could in principle raise its per-step reward by remaining in familiar early states rather than progressing, and a valid recovery manoeuvre may leave the training manifold even though it leads to success. We therefore do not use the distance as a standalone objective. We retain the large terminal penalty on failed episodes from [8], so that an episode which stays in distribution but never completes the task still receives a low return. The dense kNN distance supplies a per-step signal of how far the policy has drifted from the training distribution, while the large failure penalty preserves a strong signal of eventual task success. Together they give the value function a measure both of per-state quality and of final outcome. Section V-C shows the consequence of this assumption in practice: the method helps most where failure is visually distinct from success, and least where it is not. In Section V we compare this metric with other common OOD detection methods and show that it best predicts policy failure, motivating its use as a reward function for advantage learning. We found that task success was insensitive to the choice of k , so we used $k = 10$ as a balanced value.

The choice of encoder f determines the space in which this distance is calculated. In Section V we also compare different alternatives and find that a frozen SigLIP [39] embedding produced the best results. It is important to note that this is not a base SigLIP model, but the same fine-tuned vision encoder as the base VLA.

B. Architecture

Our overall architecture is shown in Fig. 2. It is largely similar to [8], with modifications or additions where details were unspecified in the original paper.

Value function. The value function V_ϕ consists of a pre-trained Gemma3 [40], [41] 270M parameter language model and a 400M parameter SigLIP vision encoder. To predict values, we pass in the same image, language and state conditioning c that the policy observes, and then pass the final output token through a 2-layer MLP head. Instead of predicting a continuous target, we follow [42] and predict a categorical distribution over discretised return bins, and train this with a cross-entropy loss with the targets from

(2). It is worth noting that we also tried the more complex HL-Gauss loss from [42] but found it provided no benefit over cross-entropy.

Advantage-conditioned policy. Our VLA is built from a fine-tuned $\pi_{0.5}$ [4] model, which, following π_0 [3], uses the PaliGemma [43] 3B parameter VLM along with a 300M parameter flow-matching action expert. We inject the binary advantage label into the VLM backbone by simply appending the text “Advantage: positive” or “Advantage: negative” to the end of the tokenised language input depending on the threshold value. We also follow $\pi_{0.6}^*$ and set this threshold so that 30% of the advantage labels are positive. In the unconditional case, nothing is appended. We obtained the best results by fine-tuning only the action expert and freezing the backbone. At inference the policy is then conditioned with the positive advantage label, optionally with guidance using (3).

C. Training

Training DistAL given a deployment dataset $\mathcal{D}_{deploy} = \{(o_t, a_t)\}_{t=0}^T$ proceeds in three steps:

- 1) **Score** every deployment observation with (4) against the training-set index $\mathcal{I}$, and assign the per-step reward $r_t = -s_k(o_t)$.
- 2) **Fit** the value function V_ϕ onto the empirical return $G_t = \sum_{k=0}^{T-t} r_{t+k}$ (2), then label each timestep with $y_t = \mathbb{1}[\hat{A}_t > \kappa]$ where $\hat{A}_t = G_t - V_\phi(o_t)$.
- 3) **Fine-tune** the VLA with the advantage label y_t , dropping it to $\emptyset$ with probability p at training time. At inference the policy is steered toward high-advantage actions via the classifier-free guidance velocity of (3).

V. EXPERIMENTAL RESULTS

A. OOD detection as a failure predictor

We first ask which metric best predicts task success in order to motivate the choice of reward function and encoding method for advantage conditioning.

Setup. We collect rollouts from the base $\pi_{0.5}$ policy on LIBERO, LIBERO-plus, and our two bi-manual hardware tasks, each labelled with a binary success/failure outcome. For every observation o_t we compute each method’s per-step score, and aggregate it to a trajectory-level score by taking the mean. Max aggregation was also tested but was found to perform worse across the board. We report AUROC scores against the success label, so a random score gives 0.5 and a perfect detector gives 1.0. Scores less than 0.5 indicate the classification is inverted. We report both per-trajectory and per-timestep AUROC.

Methods. We compare five detectors chosen to disentangle three axes: where in the VLA the features are tapped (row 1 vs. 2), the distance metric (row 1 vs. 3), and generative/action-space alternatives (rows 4–5):

- 1) **kNN – SigLIP features (pre-LM).** Equation (4) with the SigLIP vision encoder features of the base VLA, taken *before* the language model. Our proposed reward.
- 2) **kNN – image output tokens (post-LM).** Same kNN score, but computed on the image-token activations at

TABLE I
PER-TRAJECTORY AUROC OF EACH OOD DETECTOR AT PREDICTING ROLLOUT FAILURE. HIGHER IS BETTER; RANDOM IS 0.5.

Method	LIBERO	LIBERO+	Pen lid	Ethernet	Mean
kNN, SigLIP (ours)	0.82	0.84	0.66	0.94	0.82
kNN, LM image tokens	0.31	0.53	0.66	0.76	0.57
Mahalanobis, SigLIP	0.64	0.75	0.59	0.86	0.71
VAE likelihood	0.71	0.80	0.70	0.94	0.79
Action-chunk variance	0.60	0.68	0.51	0.54	0.58

TABLE II
PER-TIMESTEP AUROC OF EACH DETECTOR.

Method	LIBERO	LIBERO+	Pen lid	Ethernet	Mean
kNN, SigLIP (ours)	0.70	0.76	0.56	0.65	0.67
kNN, LM image tokens	0.40	0.53	0.56	0.56	0.51
Mahalanobis, SigLIP	0.57	0.64	0.52	0.58	0.58
VAE likelihood	0.64	0.73	0.59	0.64	0.65
Action-chunk variance	0.52	0.56	0.45	0.49	0.51

the *output* of the VLA language model, to test whether LM-contextualised features carry a stronger failure signal than the raw vision features.

- 3) **Mahalanobis – SigLIP features.** Class-conditional Mahalanobis distance [37] in the same SigLIP feature space as row 1, to ablate the distance metric.
- 4) **VAE likelihood.** Negative log-likelihood under a VAE trained on the same SigLIP features over $\mathcal{D}_{train}$, as a generative-model alternative to nearest-neighbour scoring.
- 5) **Action-chunk variance.** Per-step standard deviation across 16 flow-matching samples drawn from the policy for the same context c .

AUROC Results. Table I reports per-trajectory AUROC and Table II reports per-timestep AUROC. The per-timestep score is the more relevant of the two, since the value function is fitted to per-step rewards and must assign credit locally rather than classify whole episodes. Our kNN distance on the base VLA’s pre-LM SigLIP features is the strongest detector on both measures (0.82 per-trajectory and 0.67 per-timestep mean), ahead of the VAE likelihood (0.79 and 0.65), the only baseline that stays competitive across all four settings. Mahalanobis distance in the same feature space trails (0.71): the SigLIP space carries the failure signal, but the kNN metric exploits it more effectively than a class-conditional Gaussian fit. The post-LM image tokens are far less reliable (0.57 mean, even inverting to 0.31 on LIBERO), indicating that language-model contextualisation washes out rather than sharpens the signal. The pen-lid task is the hardest setting for every detector: its failures involve a correct grasp that slips, which is hard to detect visually. It is also the one place our method is not top, trailing the VAE by 0.04 (0.66 vs. 0.70); the more separable ethernet task lets our kNN distance tie for the best per-trajectory score (0.94). Overall, our method retains the best mean on both measures. The per-timestep AUROCs preserve the same ordering as the per-trajectory scores at uniformly lower values, as expected from noisier

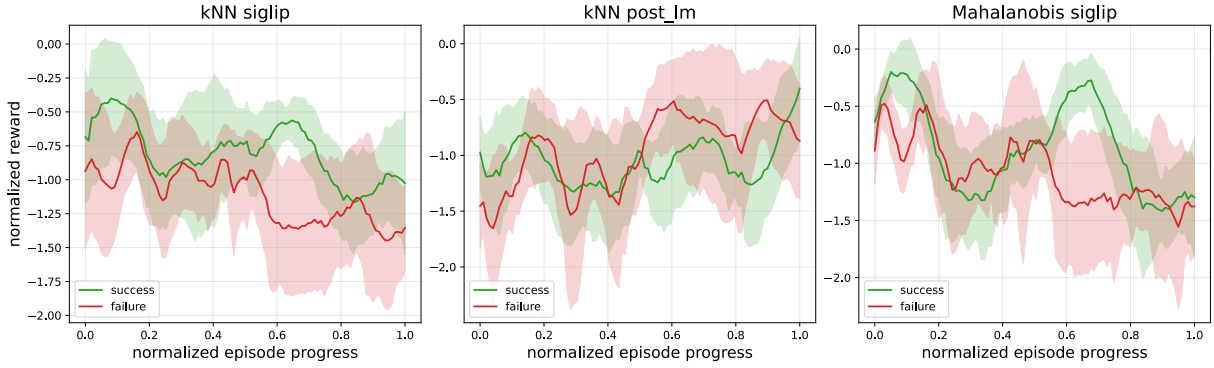


Fig. 3. Per-step rewards along a successful and a failed rollout of the same task, averaged over 10 random episodes of each and min-max normalised to $[0, 1]$ per method. The kNN score separates the two outcomes far more cleanly than the other two embedding-based detectors, which repeatedly cross over – hence its much higher AUROC. All scores drift negatively as the episode progresses, likely because final states are more diverse than initial ones.

TABLE III
SUCCESS RATE (%) ON LIBERO-10 AND PER-PERTURBATION ON LIBERO-PLUS.

Method	LIBERO-10	LIBERO-plus (per perturbation type)							Mean
		Layout	Camera	Light	Init.	Noise	Lang.	Texture	
Base $\pi_{0.5}$	94.8	77.8	41.5	96.0	65.3	59.8	76.6	89.3	75.1
SFT (success-filtered)	94.4	77.5	41.2	95.8	66.7	59.5	73.6	90.0	74.8
RECAP-style binary reward	95.0	64.0	61.9	95.6	67.3	76.1	65.2	92.3	77.2
DistAL (ours)	96.2	64.0	66.7	93.3	67.3	73.9	65.6	100.0	78.4

per-frame labels: an individual frame early in a rollout that eventually fails is often indistinguishable from one in a rollout that succeeds. The pen-lid task is again the weak point, with a per-timestep AUROC of 0.56 that is close to chance, and we return to this when interpreting the hardware results.

Plotting the per-step scores along successful and failed rollouts (Fig. 3) illustrates why the kNN distance wins, with a much cleaner separation between successful and failed rollouts than the other detectors. This figure is also the most direct evidence that the signal is usable for local credit assignment: the gap between the two curves opens progressively through the episode rather than only at the terminal frame, so the value function receives a graded, timed signal rather than a delayed one. This is the empirical motivation for using the kNN feature distance as the reward in Section IV-A; the remaining experiments in this section evaluate the downstream advantage-conditioned policy that results.

B. Simulation experiments

Having established that the kNN feature distance is a strong predictor of failure, we now evaluate whether using it as the reward function for advantage conditioning translates into improved downstream task performance.

Setup. We evaluate on LIBERO-10 (also called LIBERO-Long, the ten long-horizon tasks of LIBERO-100 [44]) and on LIBERO-plus [45]. We omit the LIBERO-Spatial, -Object, and -Goal suites as $\pi_{0.5}$ already saturates them, leaving no headroom to measure improvements. For LIBERO-plus we report per-perturbation type rather than per-suite, to reveal which robustness axes the advantage signal actually improves. We start from the same fine-tuned $\pi_{0.5}$ base policy and collect

a fixed deployment dataset $\mathcal{D}_{\text{deploy}}$ of 50 rollouts per task with success/failure labels. All advantage-conditioning methods are trained on the same dataset to isolate the effect of the reward function. We report mean success rate over 3 seeds, each with 50 episodes per task.

Baselines. We compare DistAL against three points of comparison chosen to isolate the contribution of the dense distance reward:

- 1) **Base $\pi_{0.5}$.** The fine-tuned base policy used to collect $\mathcal{D}_{\text{deploy}}$, without any advantage conditioning.
- 2) **SFT (success-filtered).** The base policy further fine-tuned on only the successful trajectories of $\mathcal{D}_{\text{deploy}}$.
- 3) **RECAP-style binary reward [8].** The same pipeline as DistAL but with the sparse success/failure reward and per-step penalty used by prior work.

Results. Table III reports success rates per task on LIBERO-10 and per perturbation type on LIBERO-plus. DistAL is the strongest method on every aggregate: it achieves the best LIBERO-10 score (96.2%) and the highest overall mean across LIBERO-10 and LIBERO-plus (78.4%, +3.3 over base). Most of that gain comes from advantage conditioning itself, which the binary-reward baseline shares; the dense distance reward adds a further +1.2 on both aggregates. This margin is consistent in sign but small relative to seed-to-seed variation, so aggregate simulation success alone does not separate the two rewards. Where they do separate is on the individual perturbation types below and on hardware, where the ethernet task shows the largest advantage of the dense reward in the paper. SFT, by contrast, is indistinguishable from the base policy (−0.3 on the overall mean): re-training

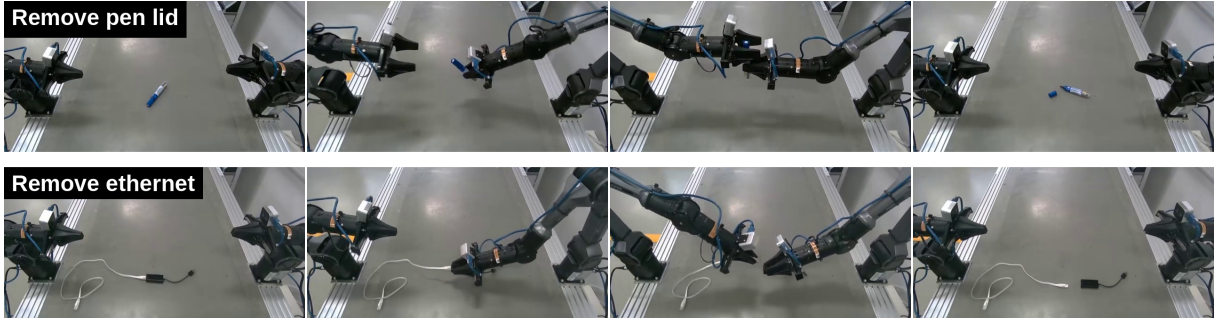


Fig. 4. Keyframes of the two bi-manual Piper tasks: *remove pen lid* (top) and *remove ethernet cable* (bottom).

on the successful slice of $\mathcal{D}_{\text{deploy}}$ extracts no additional signal, so the gains of the advantage-conditioned methods come from the value function rather than from the data subset.

The per-perturbation breakdown reveals where each reward shape pays off. Both advantage-conditioning methods deliver their largest gains on the perturbations that the base policy handles worst: *Camera* (+25.2 for DistAL and +20.4 for the binary reward over base), *Noise* (+14.1 and +16.3), and *Texture* (+10.7 and +3.0), likely because they incur the largest visual shift from the training distribution and hence are most suited to DistAL’s reward function. Our approach also widens the margin most clearly on *Camera* and *Texture*, where it reaches a perfect 100.0%. The picture is not uniformly positive. On *Layout* and *Language* both advantage-conditioned methods regress substantially relative to base (77.8 $\rightarrow$ 64.0 and 76.6 $\rightarrow$ 65.6 for DistAL), and on *Light* and *Noise* DistAL is 2.3 and 2.2 points below the binary reward. The *Layout* and *Language* regressions are almost identical for the two rewards (64.0 vs. 64.0 and 65.6 vs. 65.2), which indicates they are a property of advantage conditioning on this deployment dataset rather than of the distance reward. We believe the cause is that these perturbations leave the visual scene close to the training distribution while changing the correct behaviour, so neither reward can separate success from failure early, and the conditioning trades in-distribution performance for recovery on visually shifted observations. The *Light* and *Noise* deficits are specific to DistAL and are within seed-to-seed variation, but they are consistent with the distance reward penalising benign visual shift that does not actually threaten the task.

The comparison between DistAL and the binary-reward baseline isolates the contribution of the dense distance signal. DistAL wins on 5 of the 8 columns and matches on a sixth, with its largest individual gains coming on *Camera* (+4.8) and *Texture* (+7.7), and loses on the two noted above. The pattern, gains where the perturbation moves the observation away from the training distribution and losses where it does not, is what the reward’s assumption predicts. The next section evaluates whether the gains transfer to real hardware.

C. Hardware experiments

We now evaluate whether the gains observed in simulation transfer to a real bi-manual platform, where contact dynamics and control are substantially harder than in LIBERO.

TABLE IV
SUCCESS RATE (%) ON THE TWO BI-MANUAL PIPER TASKS, AVERAGED OVER 50 ROLLOUTS PER TASK. HIGHER IS BETTER.

Method	Pen lid	Ethernet	Mean
Base $\pi_{0.5}$	42.0	46.0	44.0
RECAP-style binary reward	48.0	72.0	60.0
DistAL (ours)	52.0	87.0	69.5

Setup. We deploy on a bi-manual setup consisting of two Piper arms, with a wrist camera on each arm and an external scene camera. We evaluate on two contact-rich, dexterous tasks: *remove pen lid*, which requires one arm to stabilise the pen body while the other grasps and pulls the lid clear; and *remove ethernet cable*, which requires one arm to brace the port while the other grips and unplugs the cable (Fig. 4). We collect 50 tele-operated demonstrations per task to fine-tune the base $\pi_{0.5}$, then a deployment dataset $\mathcal{D}_{\text{deploy}}$ of 50 more rollouts with success labels. As in simulation, all advantage-conditioning methods are trained on the *same* deployment dataset to isolate the effect of the reward function. We report mean success rate over 50 evaluation rollouts per task. We compare DistAL against the two most informative baselines carried over from the simulation study: the fine-tuned **base** $\pi_{0.5}$ without advantage conditioning, and the **RECAP-style reward**.

Results. Table IV reports success rates across the two tasks. On *remove ethernet cable*, DistAL reaches 87% against 46% for the base policy and 72% for the binary reward, our strongest single result. On *remove pen lid*, DistAL is again the best of the three at 52%, a 10-point gain over base, but only 4 points above the binary reward, a margin that is not statistically significant at 50 rollouts. We therefore treat the two methods as tied on this task.

This split matches the failure-prediction results of Section V-A. Ethernet failures (a cable left plugged in, an arm that misses the port) are visually obvious and the kNN distance separates them with a per-trajectory AUROC of 0.94. Pen-lid failures are a correctly formed grasp that slips under load, which is a contact event the image does not show, and the per-timestep AUROC there is 0.56. When the reward cannot see the failure it degrades to the binary reward plus noise, and the downstream policy behaves accordingly. DistAL

is therefore most useful when the task’s failure modes are visually distinguishable from its success modes, and we would expect it to give little benefit on tasks whose failures are primarily haptic.

Together with the simulation results, this supports DistAL as a low-cost modification to sparse-reward advantage conditioning that helps where its assumption holds, and identifies the visual separability of failure as the condition to check before applying it.

VI. CONCLUSION AND LIMITATIONS

We presented *Distance-based Advantage Learning* (DistAL), a modification to advantage-conditioned VLA fine-tuning that replaces the sparse success/failure reward with a dense per-step reward based on the k -nearest-neighbours distance to the base VLA’s training distribution in SigLIP feature space, keeping the rest of the pipeline fixed. We first showed that this distance is the strongest predictor of failure among a range of OOD-detection baselines at both the trajectory and per-timestep level, motivating its use as a reward function. We then showed that the resulting advantage-conditioned policy improves over the base $\pi_{0.5}$ policy on LIBERO-10, LIBERO-plus, and two bi-manual manipulation tasks on real hardware, and over the binary-reward baseline where failure is visually separable from success, most clearly on the *Camera* and *Texture* perturbations and on the ethernet task. Where it is not, on the pen-lid task and on perturbations that leave the scene visually unchanged, the two rewards are indistinguishable at our sample sizes.

DistAL inherits the assumptions of advantage conditioning: it relies on its own deployment data and cannot improve a policy until that policy already attains the baseline success rate needed to produce a meaningful signal, and what this level is remains an open question. Two failure modes stem from the reward itself. First, the score is purely image-based and ignores force, contact stability, robot state and language, which limited performance on the *remove pen lid* task, where the failure is a slip the camera cannot see. Second, because the reward favours proximity to the base VLA’s training distribution, a policy can in principle raise its reward by staying in familiar states rather than solving the task, and may be penalised for a novel but successful recovery; we mitigate this with the terminal penalty and by bounding the reward (Section IV-A) but have not characterised it beyond the per-perturbation results in Section V-B. Our evaluation is also limited in scope: a single base VLA ($\pi_{0.5}$), two real-world tasks, the LIBERO suites, and three training seeds per configuration, so the smaller differences we report are not statistically resolved and generality across embodiments and task families remains to be established. We have not ablated the terminal penalty, the reward normalisation, or the downstream effect of the alternative OOD scores in Table I. Promising future directions include those ablations, incorporating force into the distance score, combining the distance with an explicit task-progress signal, characterising the minimum success rate required for improvement, and performing multiple rounds of advantage conditioning to compare how the methods scale.

ACKNOWLEDGMENT

Generative AI tools (Claude, Anthropic; ChatGPT, OpenAI) were used throughout the preparation of this work, including drafting and editing text in all sections, generating and debugging code for experiments and figures, and assisting with the analysis of results. All AI-generated content was reviewed and verified by the authors, who take full responsibility for the final manuscript.

REFERENCES

- [1] A. Brohan, N. Brown, J. Carbajal *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” *arXiv preprint arXiv:2307.15818*, 2023.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti *et al.*, “OpenVLA: An open-source vision-language-action model,” *arXiv preprint arXiv:2406.09246*, 2024.
- [3] K. Black, N. Brown, D. Driess *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” *arXiv preprint arXiv:2410.24164*, 2024.
- [4] Physical Intelligence, K. Black, N. Brown *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [5] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” *arXiv preprint arXiv:2210.02747*, 2022.
- [6] A. Brohan, N. Brown, J. Carbajal *et al.*, “RT-1: Robotics transformer for real-world control at scale,” *arXiv preprint arXiv:2212.06817*, 2022.
- [7] J. Luo, C. Xu, J. Wu, and S. Levine, “Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning,” *arXiv preprint arXiv:2410.21845*, 2024.
- [8] Physical Intelligence *et al.*, “ $\pi_{0.6}^*$: A VLA that learns from experience,” *arXiv preprint arXiv:2511.14759*, 2025.
- [9] H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu, “Robot learning on the job: Human-in-the-loop autonomy and learning during deployment,” *arXiv preprint arXiv:2211.08416*, 2022.
- [10] H. Liu, Y. Zhang, V. Betala, E. Zhang, J. Liu, C. Ding, and Y. Zhu, “Multi-task interactive robot fleet learning with visual world models,” *arXiv preprint arXiv:2410.22689*, 2024.
- [11] M. S. Mark, T. Gao, G. G. Sampaio *et al.*, “Policy agnostic RL: Offline RL and online RL fine-tuning of any class and backbone,” *arXiv preprint arXiv:2412.06685*, 2024.
- [12] Y. Chen, S. Tian, S. Liu *et al.*, “ConRFT: A reinforced fine-tuning method for VLA models via consistency policy,” *arXiv preprint arXiv:2502.05450*, 2025.
- [13] Y. Guo, J. Zhang, X. Chen *et al.*, “Improving vision-language-action model with online reinforcement learning,” *arXiv preprint arXiv:2501.16664*, 2025.
- [14] S. Tan, K. Dou, Y. Zhao, and P. Krähenbühl, “Interactive post-training for vision-language-action models,” *arXiv preprint arXiv:2505.17016*, 2025.
- [15] G. Lu, W. Guo, C. Zhang, Y. Zhou, H. Jiang, Z. Gao, Y. Tang, and Z. Wang, “VLA-RL: Towards masterful and general robotic manipulation with scalable reinforcement learning,” *arXiv preprint arXiv:2505.18719*, 2025.
- [16] C. Xu, Q. Li, J. Luo, and S. Levine, “RLDG: Robotic generalist policy distillation via reinforcement learning,” *arXiv preprint arXiv:2412.09858*, 2024.
- [17] C. Xu, J. T. Springenberg, M. Equi *et al.*, “RL Token: Bootstrapping online RL with vision-language-action models,” *arXiv preprint arXiv:2604.23073*, 2026.
- [18] K. Frans, S. Park, P. Abbeel, and S. Levine, “Diffusion guidance is a controllable policy improvement operator,” *arXiv preprint arXiv:2505.23458*, 2025.
- [19] J. Ho and T. Salimans, “Classifier-free diffusion guidance,” *arXiv preprint arXiv:2207.12598*, 2022.
- [20] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” *arXiv preprint arXiv:2502.19645*, 2025.
- [21] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *arXiv preprint arXiv:2303.04137*, 2023.

- [22] Octo Model Team, D. Ghosh, H. Walke *et al.*, “Octo: An open-source generalist robot policy,” *arXiv preprint arXiv:2405.12213*, 2024.
- [23] S. Liu, L. Wu, B. Li *et al.*, “RDT-1B: A diffusion foundation model for bimanual manipulation,” *arXiv preprint arXiv:2410.07864*, 2024.
- [24] NVIDIA *et al.*, “GR00T N1: An open foundation model for generalist humanoid robots,” *arXiv preprint arXiv:2503.14734*, 2025.
- [25] Q. Li, Y. Liang, Z. Wang *et al.*, “CogACT: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation,” *arXiv preprint arXiv:2411.19650*, 2024.
- [26] A. Kumar, A. Zhou, G. Tucker, and S. Levine, “Conservative Q-learning for offline reinforcement learning,” *arXiv preprint arXiv:2006.04779*, 2020.
- [27] I. Kostrikov, A. Nair, and S. Levine, “Offline reinforcement learning with implicit Q-learning,” *arXiv preprint arXiv:2110.06169*, 2021.
- [28] X. B. Peng, A. Kumar, G. Zhang, and S. Levine, “Advantage-weighted regression: Simple and scalable off-policy reinforcement learning,” *arXiv preprint arXiv:1910.00177*, 2019.
- [29] L. Chen, K. Lu, A. Rajeswaran *et al.*, “Decision transformer: Reinforcement learning via sequence modeling,” *arXiv preprint arXiv:2106.01345*, 2021.
- [30] M. Nakamoto, O. Mees, A. Kumar, and S. Levine, “Steering your generalists: Improving robotic foundation models via value guidance,” *arXiv preprint arXiv:2410.13816*, 2024.
- [31] R. Sinha, A. Elhafsi, C. Agia, M. Foutter, E. Schmerling, and M. Pavone, “Real-time anomaly detection and reactive planning with large language models,” *arXiv preprint arXiv:2407.08735*, 2024.
- [32] C. Agia, R. Sinha, J. Yang *et al.*, “Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress,” *arXiv preprint arXiv:2410.04640*, 2024.
- [33] C. Agia, T. Migimatsu, J. Wu, and J. Bohg, “STAP: Sequencing task-agnostic policies,” *arXiv preprint arXiv:2210.12250*, 2022.
- [34] C. Xu *et al.*, “Can we detect failures without failure data? uncertainty-aware runtime failure detection for imitation learning policies,” *arXiv preprint arXiv:2503.08558*, 2025.
- [35] A. Z. Ren, A. Dixit, A. Bodrova *et al.*, “Robots that ask for help: Uncertainty alignment for large language model planners,” *arXiv preprint arXiv:2307.01928*, 2023.
- [36] Y. Sun, Y. Ming, X. Zhu, and Y. Li, “Out-of-distribution detection with deep nearest neighbors,” *arXiv preprint arXiv:2204.06507*, 2022.
- [37] K. Lee, K. Lee, H. Lee, and J. Shin, “A simple unified framework for detecting out-of-distribution samples and adversarial attacks,” *arXiv preprint arXiv:1807.03888*, 2018.
- [38] B. Lakshminarayanan, A. Pritzel, and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” *arXiv preprint arXiv:1612.01474*, 2016.
- [39] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, “Sigmoid loss for language image pre-training,” *arXiv preprint arXiv:2303.15343*, 2023.
- [40] Gemma Team *et al.*, “Gemma 3 technical report,” *arXiv preprint arXiv:2503.19786*, 2025.
- [41] Google Developers, “Introducing Gemma 3 270M: The compact model for hyper-efficient AI,” <https://developers.googleblog.com/en/introducing-gemma-3-270m/>, 2025, model: <https://huggingface.co/google/gemma-3-270m>.
- [42] J. Farebrother, J. Orbay, Q. Vuong *et al.*, “Stop regressing: Training value functions via classification for scalable deep RL,” *arXiv preprint arXiv:2403.03950*, 2024.
- [43] L. Beyer, A. Steiner, A. S. Pinto *et al.*, “PaliGemma: A versatile 3B VLM for transfer,” *arXiv preprint arXiv:2407.07726*, 2024.
- [44] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” *arXiv preprint arXiv:2306.03310*, 2023.
- [45] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu, “LIBERO-plus: In-depth robustness analysis of vision-language-action models,” *arXiv preprint arXiv:2510.13626*, 2025.